



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y
DISEÑO DIGITAL
CARRERA DE INGENIERÍA DE SOFTWARE

APLICACIONES WEB – 2025-2026

TEMA:

GA – Práctica en clase

Gestión de Estado: Sesiones y Carrito de Compras

Implementación comparativa en PHP 8, ASP.NET Core 8 y
Java / Spring Boot 3

ESTUDIANTE:

PANAMÁ MURILLO MOISÉS ANTONIO

MATERIA:

Aplicaciones Web

Contents

1	Introduccion	2
2	Entorno y herramientas	2
3	Estructura del repositorio	2
4	Plataforma 1 — PHP 8 con \$_SESSION	2
4.1	Configuracion de la sesion segura	2
4.2	Logica de agregar, eliminar y limpiar	3
4.3	Ejecucion y verificacion	3
5	Plataforma 2 — ASP.NET Core 8 con ISession	4
5.1	Configuracion de sesiones en Program.cs	4
5.2	Controlador del carrito	4
5.3	Ejecucion y verificacion	5
6	Plataforma 3 — Java / Spring Boot 3 con HttpSession	5
6.1	Configuracion de sesion en application.yaml	5
6.2	Modelo y controlador	6
6.3	Ejecucion y verificacion	6
7	Tabla comparativa de las tres plataformas	7
8	Por que el PFC usa JWT en lugar de sesiones de servidor	7
9	Preguntas de reflexion	8
9.1	1. Balanceador de carga con multiples instancias PHP	8
9.2	2. session_unset() vs session_destroy()	8
9.3	3. Serializacion del carrito a JSON en ASP.NET Core	8
9.4	4. XSS con flag HttpOnly activo	8
9.5	5. Cierre del navegador sin logout y el accessToken JWT	8
10	Flujo de trabajo en Git/GitHub	9
10.1	Comandos utilizados	9
10.2	Historial de commits	9
10.3	Enlace al repositorio	10
11	Conclusiones	10

1 Introduccion

Esta practica implementa un **carrito de compras minimo** con tres operaciones (agregar, eliminar y limpiar productos) usando el mecanismo de **sesiones de servidor** en tres plataformas del semestre: **PHP 8** con `$_SESSION`, **ASP.NET Core 8** con `ISession`, y **Java / Spring Boot 3** con `HttpSession`. El objetivo es evidenciar las diferencias de API, configuracion y seguridad de cookies entre las tres tecnologias, y reflexionar sobre por que el Proyecto Final de Curso (PFC) reemplaza las sesiones de servidor por tokens JWT.

2 Entorno y herramientas

- **Editor:** Visual Studio Code (terminal integrada de PowerShell)
- **PHP:** XAMPP – Apache + PHP 8.2.12
- **ASP.NET Core:** SDK .NET 8.0.422 (fijado con `global.json` ante la presencia de .NET 10 en el equipo)
- **Java / Spring Boot:** JDK 21 (Temurin), Maven Wrapper (`mvnw`), Spring Boot 3.5.16
- **Sistema operativo:** Windows
- **Control de versiones:** Git + GitHub
- **Repositorio:** <https://github.com/MoisesPanama/gestion-estado-sesiones-carrito-compras>

3 Estructura del repositorio

```
gestion-estado-sesiones-carrito-compras/  
|-- php-carrito/  
| |-- configuracion.php  
| |-- index.php  
| |-- agregar.php  
| |-- eliminar.php  
| '-- limpiar.php  
|-- aspnet-carrito/  
| |-- Program.cs  
| |-- Models/Producto.cs  
| |-- Controllers/CarritoController.cs  
| '-- Views/Carrito/Index.cshtml  
|-- spring-carrito/  
| |-- src/main/resources/application.yaml  
| |-- src/main/java/ec/edu/uteq/carrito/model/Producto.java  
| |-- src/main/java/ec/edu/uteq/carrito/controller/CarritoController.java  
| '-- src/main/resources/templates/carrito/index.html  
|-- docs/  
| |-- informe_sesiones_carrito.pdf  
| '-- capturas/  
|-- .gitignore  
'-- README.md
```

4 Plataforma 1 — PHP 8 con `$_SESSION`

4.1 Configuracion de la sesion segura

```
<?php  
// Configurar ANTES de session_start()  
ini_set('session.cookie_httponly', '1'); // HttpOnly  
// ini_set('session.cookie_secure', '1'); // -- desactivado en local, activar en produccion  
ini_set('session.cookie_samesite', 'Strict'); // Anti-CSRF  
ini_set('session.use_strict_mode', '1'); // IDs regenerados
```

```
ini_set('session.gc_maxlifetime', '1800'); // 30 min timeout

session_start();

// Regenerar ID al iniciar sesion (previene Session Fixation)
if (!isset($_SESSION['iniciada'])) {
    session_regenerate_id(true);
    $_SESSION['iniciada'] = true;
}
```

4.2 Logica de agregar, eliminar y limpiar

```
// agregar.php (fragmento)
if ($nombre !== null && array_key_exists($nombre, $precios)) {
    if (!isset($_SESSION['carrito'])) {
        $_SESSION['carrito'] = [];
    }
    $_SESSION['carrito'][] = [
        'nombre' => $nombre,
        'precio' => $precios[$nombre],
    ];
}
header('Location: index.php');
exit;
```

```
// eliminar.php (fragmento)
$indice = filter_input(INPUT_POST, 'indice', FILTER_VALIDATE_INT);
if ($indice !== false && $indice !== null
    && isset($_SESSION['carrito'][$indice])) {
    array_splice($_SESSION['carrito'], $indice, 1);
}
```

```
// limpiar.php (fragmento)
$_SESSION['carrito'] = []; // Opcion A: vaciar, mantiene la sesion
// session_unset(); session_destroy(); // Opcion B: destruir la sesion completa
```

4.3 Ejecucion y verificacion

El proyecto se sirvio con XAMPP (Apache + PHP 8.2.12) mediante un enlace simbolico desde C:\xampp\htdocs\carrito hacia la carpeta php-carrito/ del repositorio, en <http://localhost/carrito/index.php>. Se verifico que el carrito persiste al agregar, eliminar y vaciar productos, y que la cookie PHPSESSID tiene el flag **HttpOnly** activo en DevTools.

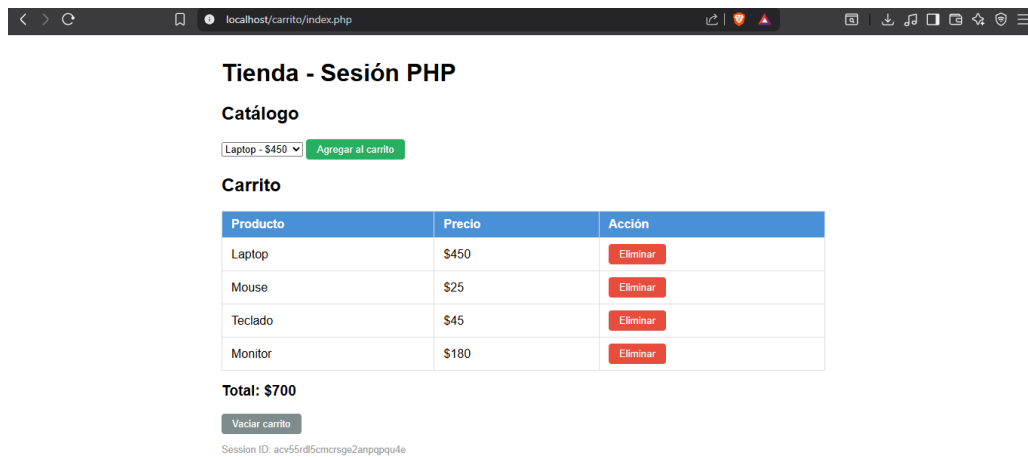


Figure 1: Carrito PHP en el navegador con productos agregados y el total calculado.

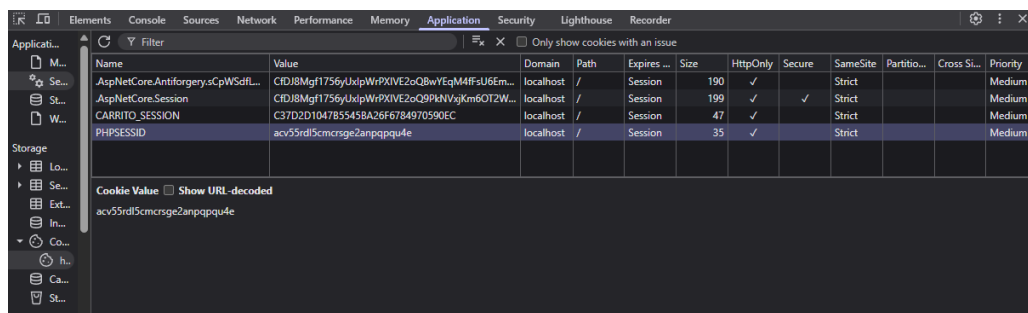


Figure 2: Cookie PHPSESSID en DevTools mostrando el flag HttpOnly activo.

5 Plataforma 2 — ASP.NET Core 8 con ISession

5.1 Configuración de sesiones en Program.cs

```
builder.Services.AddDistributedMemoryCache();

builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true; // flag HttpOnly
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always; // flag Secure
    options.Cookie.SameSite = SameSiteMode.Strict; // flag SameSite
    options.Cookie.IsEssential = true;
});
// ...
app.UseSession(); // ANTES de MapControllers
```

5.2 Controlador del carrito

```
private List<Producto> ObtenerCarrito()
{
    var json = HttpContext.Session.GetString(CLAVE_CARRITO);
    return json is null
        ? new List<Producto>()
        : JsonSerializer.Deserialize<List<Producto>>(json!);
}
```

```

}

private void GuardarCarrito(List<Producto> carrito)
{
    HttpContext.Session.SetString(
        CLAVE_CARRITO, JsonSerializer.Serialize(carrito));
}

```

A diferencia de PHP, `ISession` solo admite `string` o `byte[]`, por lo que el carrito se serializa manualmente a JSON antes de guardarse en la sesion.

5.3 Ejecucion y verificacion

Se genero el proyecto con `dotnet new mvc`, fijando el SDK en version 8 mediante un archivo `global.json` (el equipo tenia tambien instalado el SDK 10). El servidor se ejecuto con `dotnet run -launch-profile https`, quedando disponible en <https://localhost:7074/Carrito>. Se verifico en DevTools que la cookie `.AspNetCore.Session` tiene los tres flags de seguridad activos: `HttpOnly`, `Secure` y `SameSite=Strict`.

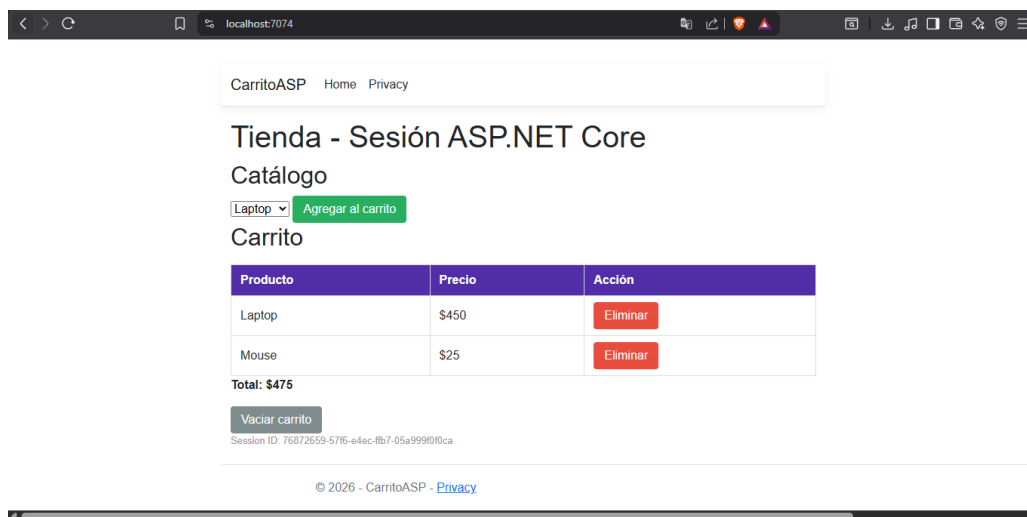


Figure 3: Carrito ASP.NET Core en el navegador con productos agregados.

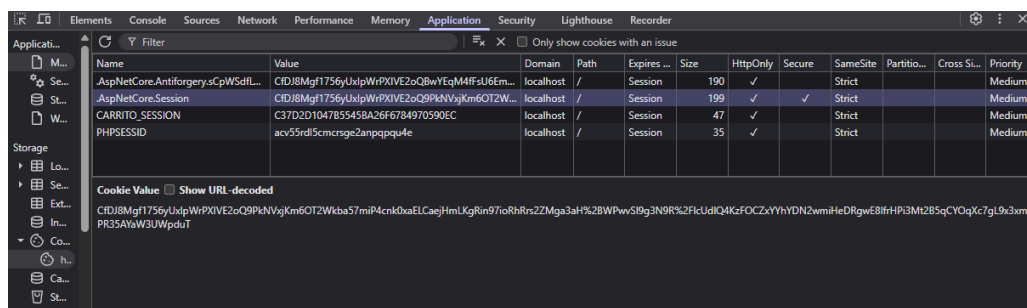


Figure 4: Cookie `.AspNetCore.Session` en DevTools mostrando `HttpOnly`, `Secure` y `SameSite=Strict`.

6 Plataforma 3 — Java / Spring Boot 3 con HttpSession

6.1 Configuracion de sesion en application.yaml

```
server:
  port: 8080
  servlet:
    session:
      timeout: 30m
      cookie:
        http-only: true # flag HttpOnly
        secure: false # true en produccion (HTTPS)
        same-site: strict # flag SameSite=Strict
        name: CARRITO_SESSION
```

6.2 Modelo y controlador

```
public record Producto(String nombre, double precio)
    implements Serializable {
    private static final long serialVersionUID = 1L;
}
```

```
@SuppressWarnings("unchecked")
private List<Producto> obtenerCarrito(HttpSession session) {
    List<Producto> carrito =
        (List<Producto>) session.getAttribute(CLAVE);
    if (carrito == null) {
        carrito = new ArrayList<>();
        session.setAttribute(CLAVE, carrito);
    }
    return carrito;
}
```

A diferencia de ASP.NET Core, `HttpSession` permite guardar objetos complejos directamente con `setAttribute()`, sin necesidad de serializarlos manualmente a JSON; sin embargo, la clase debe implementar `Serializable` para evitar una `NotSerializableException`.

6.3 Ejecucion y verificacion

El proyecto se genero desde Spring Initializr (Maven, Java 21, Spring Boot 3.5.16, dependencias Spring Web y Thymeleaf) y se ejecuto con el Maven Wrapper: `./mvnw spring-boot:run`, quedando disponible en <http://localhost:8080/carrito>. Se verifico en DevTools que la cookie `CARRITO_SESSION` tiene el flag **HttpOnly** activo.

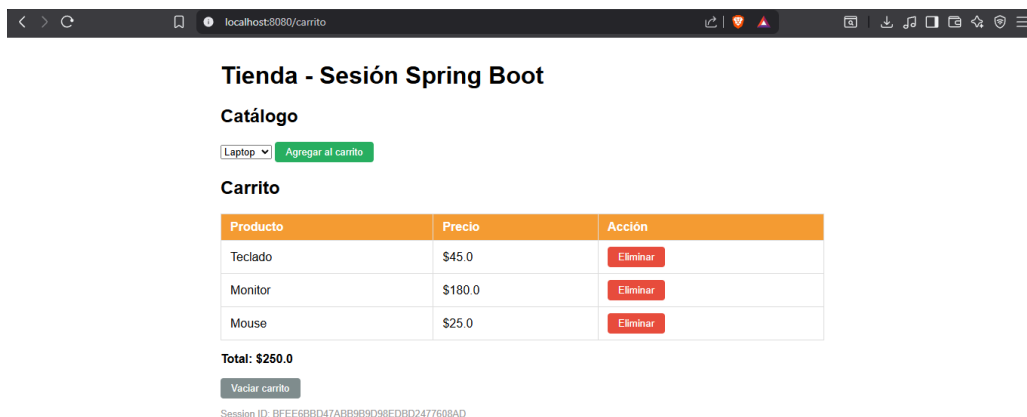


Figure 5: Carrito Spring Boot en el navegador con productos agregados.

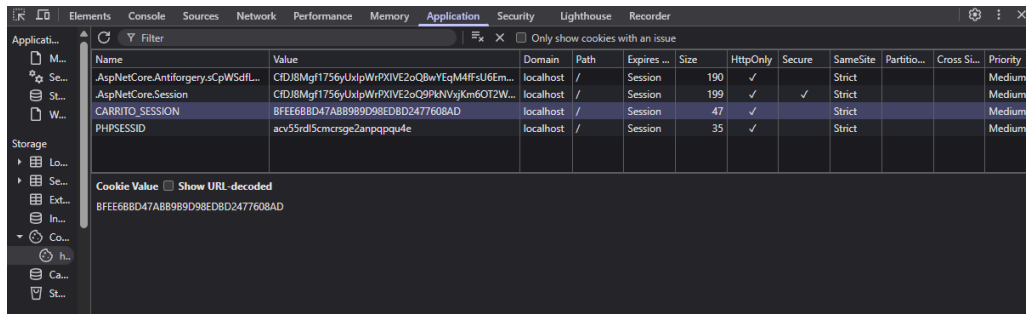


Figure 6: Cookie CARRITO_SESSION en DevTools mostrando el flag HttpOnly activo.

7 Tabla comparativa de las tres plataformas

Criterio	PHP 8	ASP.NET Core 8	Java / Spring Boot 3
Iniciar sesion	<code>session_start()</code> al inicio de cada script	<code>AddSession()</code> en <code>Program.cs</code> <code>UseSession()</code>	Automatico con Spring MVC; <code>HttpSession</code> inyectada
Guardar dato	<code>\$_SESSION[k] = valor;</code> , nativo	Serializar a JSON + <code>SetString()</code>	<code>setAttribute(k, obj)</code> directo
Leer dato	<code>\$_SESSION[k] ?? []</code>	<code>GetString()</code> + deserializar	<code>getAttribute(k)</code> con cast
Destruir sesion	<code>session_destroy()</code>	<code>Session.Clear()</code>	<code>session.invalidate()</code>
Nombre cookie	PHPSESSID	<code>.AspNetCore.Session</code>	JSESSIONID (o personalizado en YAML)
HttpOnly	<code>cookie_httponly = 1</code>	<code>Cookie.HttpOnly = true</code>	<code>http-only: true</code>
Secure	<code>cookie_secure = 1</code>	<code>SecurePolicy.Always</code>	<code>secure: true/false</code>
SameSite	<code>cookie_samesite = Strict</code>	<code>SameSite.Strict</code>	<code>same-site: strict</code>
Almacenamiento	Archivos en <code>/tmp</code> o Redis	Memoria, Redis o SQL Server	Memoria JVM, Redis o JDBC
Escalabilidad	Sticky sessions o Redis	Redis nativo	Spring Session + Redis

8 Por que el PFC usa JWT en lugar de sesiones de servidor

Las sesiones de servidor son adecuadas para aplicaciones web tradicionales multi-pagina, como la implementada en esta practica. Sin embargo, el Proyecto Final de Curso (PFC) utiliza una arquitectura distinta: un frontend SPA en Angular que consume una API REST de Spring Boot. En esa arquitectura, las sesiones de servidor resultan inadecuadas por tres razones:

1. **Estado en el servidor:** una sesion obliga al servidor a recordar cada usuario, complicando el escalado horizontal; un JWT es *stateless* y cualquier instancia puede validarlo sin compartir memoria.
2. **API REST y SPA:** las cookies de sesion no encajan naturalmente con APIs REST consumidas por SPAs (requieren manejo manual de CORS y credenciales); el JWT viaja en el encabezado `Authorization: Bearer` o en una cookie `HttpOnly`, compatible con CORS.

3. **Seguridad del token:** un `accessToken` JWT de corta vida (1 hora) limita la ventana de exposición si es robado, mientras que un `refreshToken` en cookie `HttpOnly` permite renovarlo sin que JavaScript pueda leerlo, protegiendo contra XSS.

En una oración: el PFC usa JWT en cookie `HttpOnly` porque mantiene la arquitectura REST sin estado en el servidor, escala horizontalmente sin configuración adicional entre instancias, y protege el token contra ataques XSS ya que JavaScript no puede leer una cookie `HttpOnly`.

9 Preguntas de reflexion

9.1 1. Balanceador de carga con multiples instancias PHP

Si el servidor PHP tiene dos instancias detras de un balanceador y las sesiones se guardan en archivos locales (`/tmp`), cada instancia tendria su propio conjunto de sesiones, por lo que un usuario perderia su carrito si el balanceador lo redirige a una instancia distinta a la que inicio su sesion. Esto se resuelve centralizando el almacenamiento de sesiones en un backend compartido, como **Redis**, usando la extension `session.save_handler = redis` en PHP. En **Spring Boot** el mismo problema se resuelve con **Spring Session + Redis**, que reemplaza el almacenamiento en memoria de la JVM por un almacen distribuido compartido entre instancias.

9.2 2. `session_unset()` vs `session_destroy()`

`session_unset()` vacia todas las variables de la sesion actual (el array `$_SESSION` queda vacio) pero mantiene la sesion activa y la cookie valida; se usaria cuando se quiere limpiar los datos de sesion sin cerrar la sesion del usuario. Por su parte, `session_destroy()` elimina por completo los datos de sesion en el servidor (el archivo de sesion) y requiere ademas invalidar la cookie manualmente; se usaria al hacer *logout* completo, cuando se quiere terminar la sesion del usuario en su totalidad.

9.3 3. Serializacion del carrito a JSON en ASP.NET Core

En ASP.NET Core, la interfaz `ISession` solo permite almacenar tipos primitivos: `string`, `int` o `byte[]`; no admite guardar objetos complejos directamente como en Java. Por eso el carrito (una lista de objetos `Producto`) debe convertirse primero a una cadena JSON con `JsonSerializer.Serialize()` antes de guardarse con `Session.SetString()`, y deserializarse de vuelta al leerlo. En Java, `HttpSession.setAttribute()` acepta cualquier objeto que implemente `Serializable`, por lo que no es necesario convertirlo a texto manualmente.

9.4 4. XSS con flag `HttpOnly` activo

Si un atacante logra inyectar JavaScript en la pagina PHP (XSS), el flag **`HttpOnly`** le impide leer el valor de la cookie `PHPSESSID` mediante `document.cookie`, por lo que **no puede robar directamente el identificador de sesion** de esa forma. Sin embargo, aunque no pueda leer la cookie, el atacante igualmente podria ejecutar acciones **en nombre del usuario autenticado** dentro de esa misma pagina (por ejemplo, enviar formularios ocultos que agreguen o eliminen productos del carrito, o exfiltrar otros datos sensibles visibles en el DOM), ya que el navegador seguira enviando la cookie automáticamente en esas peticiones. Es decir, `HttpOnly` protege el *valor* de la cookie, pero no impide que un XSS abuse de la sesion activa mientras el usuario sigue navegando la pagina comprometida.

9.5 5. Cierre del navegador sin logout y el `accessToken` JWT

Si el usuario cierra el navegador sin hacer logout, el `accessToken` JWT sigue siendo tecnicamente valido durante el resto de su hora de vida, ya que un JWT es *stateless*: el servidor no lo invalida

automaticamente solo porque el cliente cerro la ventana (a menos que se implemente una lista de revocacion). Esto se mitiga mediante el `refreshToken` almacenado en una cookie `HttpOnly`: al tener una vida mas larga pero estar protegido de lectura por JavaScript, permite renovar el `accessToken` de forma seguridad en la siguiente visita, y su revocacion (por ejemplo mediante una *blacklist* en Redis usando el JTI del token) permite invalidar sesiones de forma centralizada si el usuario cierra sesion explicitamente o si se detecta actividad sospechosa.

10 Flujo de trabajo en Git/GitHub

10.1 Comandos utilizados

```
git clone https://github.com/MoisesPanama/gestion-estado-sesiones-carrito-compras.git
cd gestion-estado-sesiones-carrito-compras

# Estructura y gitignore
git add .
git commit -m "chore: crear estructura de carpetas y gitignore para las tres plataformas"
git push origin main

# PHP
git add .
git commit -m "feat/php): implementar carrito de compras con sesiones en PHP 8"
git push origin main

# ASP.NET Core
git add .
git commit -m "feat(aspnet): crear proyecto MVC base con .NET 8 forzado via global.json"
git commit -m "feat(aspnet): implementar carrito de compras con ISession y flags de seguridad"
git push origin main

# Spring Boot
git add .
git commit -m "feat(springboot): implementar carrito de compras con HttpSession y flags de
seguridad"
git push origin main
```

10.2 Historial de commits

Hash	Mensaje
83c94e1	Initial commit
cc20ea3	chore: crear estructura de carpetas y gitignore para las tres plataformas
6be177f	feat/php): implementar carrito de compras con sesiones en PHP 8
5c9340f	fix/php): desactivar temporalmente flag Secure para pruebas en localhost sin HTTPS
fd03b92	feat(aspnet): crear proyecto MVC base con .NET 8 forzado via global.json
250320b	feat(aspnet): implementar carrito de compras con ISession y flags de seguridad HttpOnly/Secure/SameSite
c5ab970	feat(springboot): implementar carrito de compras con HttpSession y flags de seguridad en application.yaml

10.3 Enlace al repositorio

<https://github.com/MoisesPanama/gestion-estado-sesiones-carrito-compras>

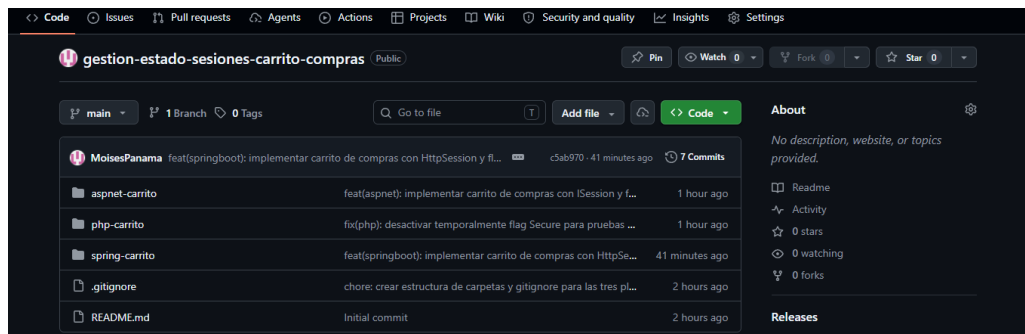


Figure 7: Repositorio en GitHub mostrando el historial de commits y las tres carpetas de plataformas.

11 Conclusiones

Esta practica permitio comparar de forma directa el mecanismo de **gestion de estado mediante sesiones de servidor** en tres plataformas distintas del semestre. Aunque la API varia (arrays nativos en PHP, serializacion JSON manual en ASP.NET Core, y objetos directos en Java), el concepto subyacente es el mismo en los tres casos: un identificador unico enviado como cookie que le permite al servidor recuperar datos asociados a un usuario entre peticiones. Los tres flags de seguridad de cookies (**HttpOnly**, **Secure** y **SameSite**) resultaron configurables de forma equivalente en las tres plataformas, y su comprension resulto clave para entender por que el PFC, al usar una arquitectura SPA + API REST, reemplaza este modelo por **JWT almacenado en cookie HttpOnly**: una solucion *stateless*, escalable horizontalmente y protegida contra XSS.